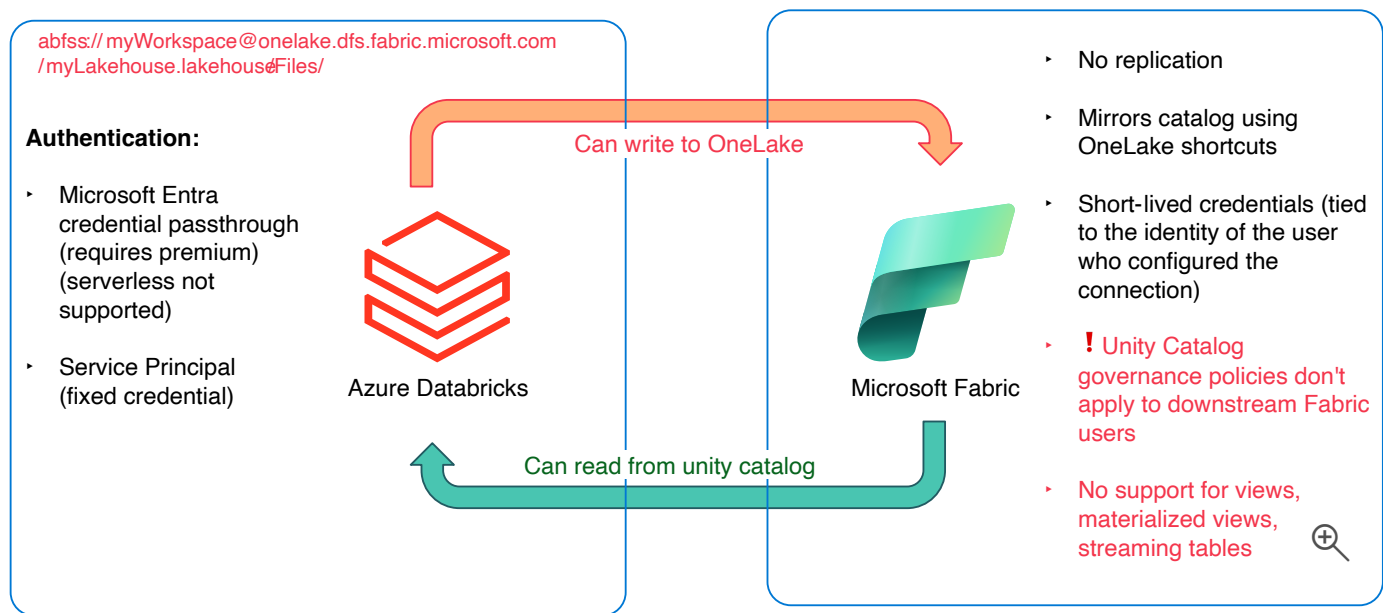


# Understand integration with Microsoft Fabric

8 minutes

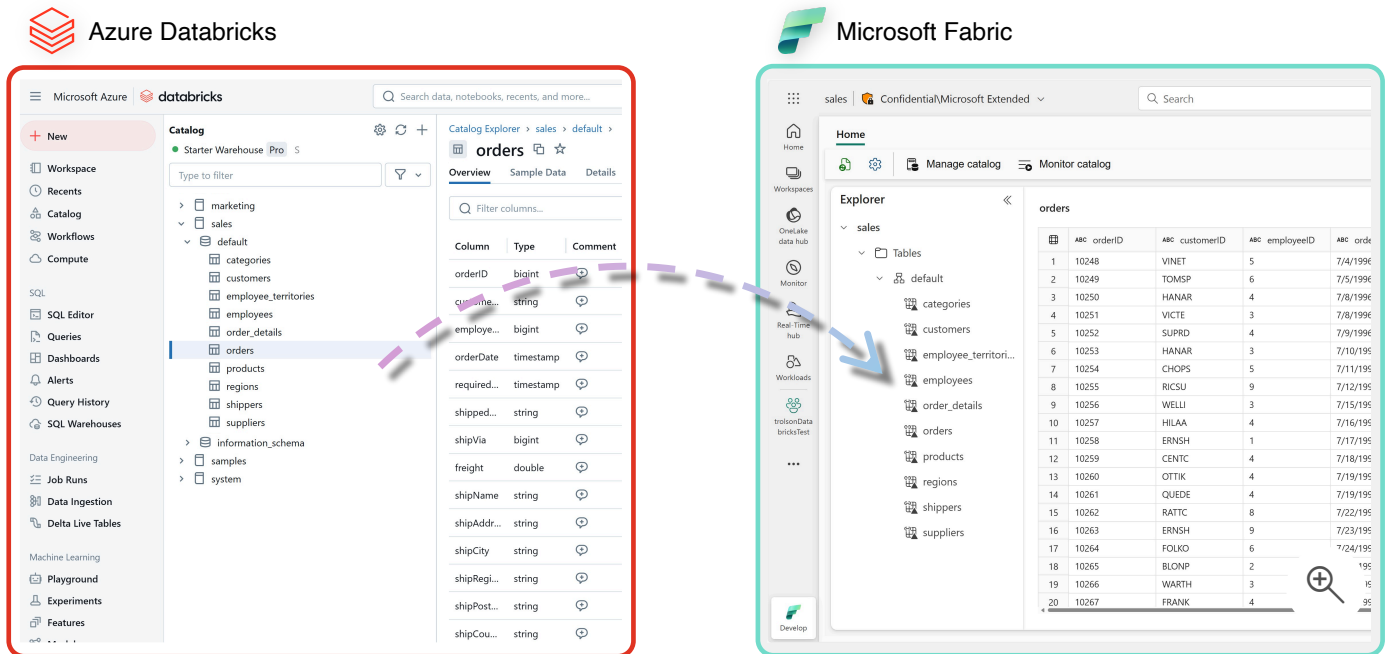
Microsoft Fabric and Azure Databricks complement each other in modern data architectures. Both platforms excel at data engineering and advanced analytics, with Azure Databricks specializing in large-scale data processing and machine learning, while Fabric offers a unified platform with integrated business intelligence and reporting capabilities. Understanding how these platforms integrate helps you design solutions that leverage the strengths of both.

The integration between Azure Databricks and Microsoft Fabric works **bidirectionally**. Fabric can **read** data registered in Unity Catalog, and Azure Databricks can **write** data to OneLake. Each integration pattern serves different scenarios and comes with specific considerations for governance and security.



## Access Unity Catalog data from Fabric

Fabric can read tables registered in Unity Catalog **without replicating the data**. When you create a **Mirrored Azure Databricks Catalog** item in Fabric, it mirrors the catalog structure and automatically creates **OneLake shortcuts** for each table. These shortcuts point to the Delta tables stored in Azure Data Lake Storage. The mirroring process doesn't copy your data. Instead, Fabric uses Unity Catalog's open APIs to obtain credentials for accessing the underlying storage paths.



The authentication mechanism relies on **short-lived credentials** that Unity Catalog provides to Fabric. These credentials refresh every hour and can be revoked through Unity Catalog at any time. This approach maintains data in a **single location** while allowing Fabric users to query it.

However, this integration has an **important security implication**. Fabric engines perform authorization using the identity of the user who configured the connection, **not the identity of users who query the data**. **Unity Catalog governance policies don't apply to downstream Fabric users**. Once a table is exposed in Fabric, any Fabric user with access to the connection can query it, regardless of Unity Catalog permissions. Evaluate this against your organization's security requirements.

The user who configures the Fabric connection must have the **EXTERNAL USE SCHEMA** privilege on the schemas in Unity Catalog that contain the tables to share. This privilege allows Unity Catalog to issue credentials for the underlying storage. After the connection is established, downstream Fabric users don't need this privilege.

Several **limitations** apply to this integration pattern. Fabric can't access views, materialized views, streaming tables, Delta Sharing catalogs, or tables with row-level filters or column masks. **Only Delta format tables are supported.** The integration doesn't work with workspaces that use private endpoints or IP access lists, and Unity Catalog lineage doesn't track operations performed in Fabric.

## Write data to OneLake from Databricks

Azure Databricks can write data directly to Microsoft Fabric lakehouses in OneLake. This integration pattern supports scenarios where you process and transform data in Databricks, then make it available in Fabric for analytics and reporting.

To connect Azure Databricks to OneLake, you use the **Azure Blob Filesystem (ABFS) driver** with OneLake endpoints. The connection path follows the format:

```
abfss://myWorkspace@onelake.dfs.fabric.microsoft.com/myLakehouse.lakehouse/Files/.
```

This path structure identifies your Fabric workspace and lakehouse, allowing Databricks to read from and write to specific locations.

You can authenticate to OneLake through **two methods**. With **Microsoft Entra credential passthrough**, user identities flow through to OneLake, but this requires a **premium Azure Databricks workspace** and proper cluster configuration with **ADLS credential passthrough enabled**. Alternatively, you can authenticate using a **service principal**, which uses a fixed credential rather than individual user identities. Service principals work with both traditional clusters and Azure Databricks serverless compute, providing more flexibility for automated workflows and job execution.

### Note

Credential passthrough is not supported with serverless compute

The workflow is straightforward. You load data in Databricks, apply transformations using Spark, and write the results to OneLake using standard Spark write operations. Once data lands in your Fabric lakehouse, it becomes immediately available to Power BI, data warehouses, and other Fabric services without additional data movement.

This integration supports **serverless compute**, allowing you to run workloads without provisioning clusters. When using serverless compute, you must use service principal authentication and ensure your code doesn't modify unsupported Spark configuration properties. The service principal needs appropriate workspace role assignments in Fabric to write data.

## Integration scenarios

These integration patterns enable several practical scenarios for organizations using both platforms. You can build **data pipelines** where Databricks handles complex transformations and machine learning workloads, then publishes curated datasets to Fabric for business analytics. Data engineers work in Azure Databricks while business analysts access the results through familiar Fabric tools.

With Fabric reading from Unity Catalog, you can make specific datasets available for reporting **without duplicating storage**. Business users create Power BI reports that query data managed in Unity Catalog, maintaining a **single source of truth**. This approach applies when you have data governance established in Unity Catalog and want to extend access to Fabric users for specific tables.

For scenarios requiring **real-time or near-real-time** analytics, Databricks streaming jobs can write processed data to OneLake continuously. Fabric workloads can then consume this data as it arrives. This pattern separates the responsibilities of data engineering and business intelligence while keeping them connected through shared storage.

When you need to combine data from multiple sources, you can use Azure Databricks to **join and transform** data from various systems, then land the integrated results in OneLake. Fabric serves as the presentation layer, providing self-service analytics on the prepared datasets. This architecture lets data engineers focus on data quality and transformation logic while business users create their own analyses.

## Key considerations

**Security and governance** are critical factors when you implement these integrations. The Fabric-to-Unity Catalog integration **bypasses Unity Catalog's fine-grained access controls** for downstream users by default. Carefully consider which tables to expose and ensure your organization accepts this security model. You can partially address this by enabling **OneLake**

**security** on the Mirrored Azure Databricks Catalog item. This two-step approach first synchronizes a Microsoft Entra group and grants it the necessary Unity Catalog privileges using Automatic Identity Management, then assigns that group a **OneLake Data Access Role** in the Fabric workspace to control access to the corresponding shortcuts. For scenarios requiring row-level filters or column masks at the individual user level, **Power BI DirectQuery** remains the recommended path, as it honors Unity Catalog permissions directly.

Performance and cost also matter. Reading Unity Catalog data from Fabric requires a running Fabric capacity for metadata scans and refreshes, which introduces additional costs. When writing from Azure Databricks to OneLake, network transfer costs apply if data crosses region boundaries. Plan your architecture to minimize unnecessary data movement and optimize for your workload patterns.

Storage architecture remains important. Both integration patterns leverage **Azure Data Lake Storage** as the underlying storage layer. Azure Databricks uses Unity Catalog to manage and govern this storage, while Fabric accesses it through OneLake abstractions. Understanding this **shared foundation** helps you design efficient solutions that avoid data duplication and maintain consistency.

The choice between integration patterns depends on your specific requirements. Use Fabric reading from Unity Catalog when you want to expose existing Databricks-managed data to Fabric users and can accept the security model. Use Azure Databricks writing to OneLake when you need to publish curated datasets specifically for Fabric consumption and want more control over what data is shared. In many architectures, you might use both patterns for different scenarios within the same organization.

---

## Next unit: Understand integration with Power BI

[< Previous](#)[Next >](#)